# İşletim Sistemleri
# (Operating Systems)

# Process Zamanlama
# (Process Scheduling)

# Çoklu Kuyruk Düzenleri:

1. **Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)**

2. **Çok Düzeyli Geri Bildirim Kuyruk Zamanlama (Multilevel Feedback Queue Scheduling)**

# Çok Düzeyli Kuyruk Zamanlama

# (Multilevel Queue Scheduling)

# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

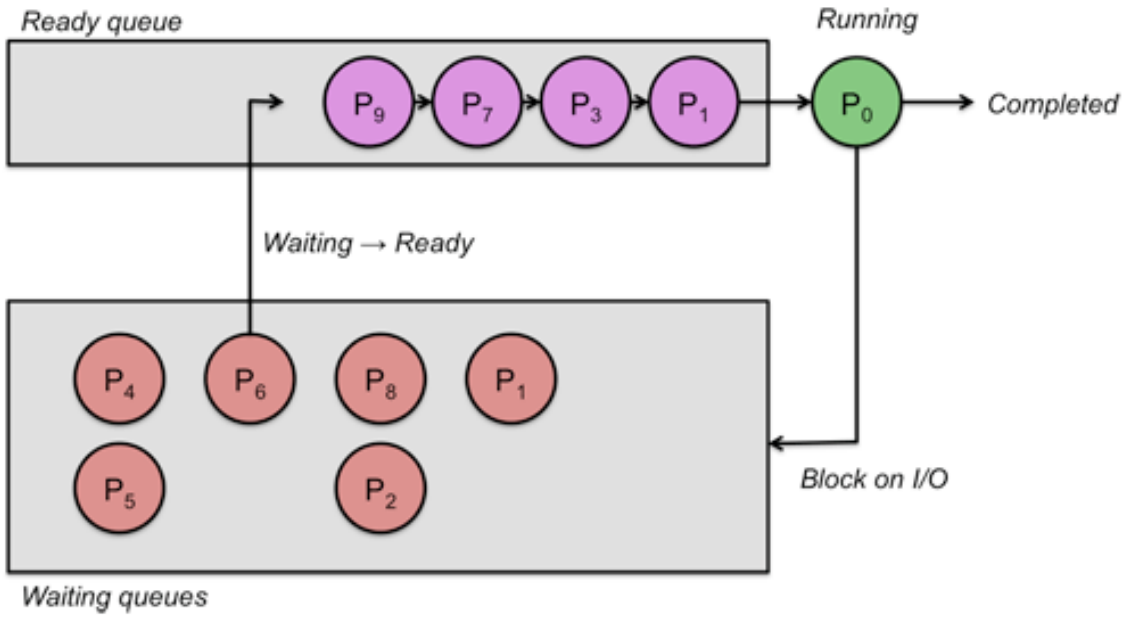
- Process'ler kabaca iki sınıfa ayrılabilir:
  - Foreground (interactive) process'ler (Yüksek öncelikli)
  - Background (batch) process'ler (Düşük öncelikli)

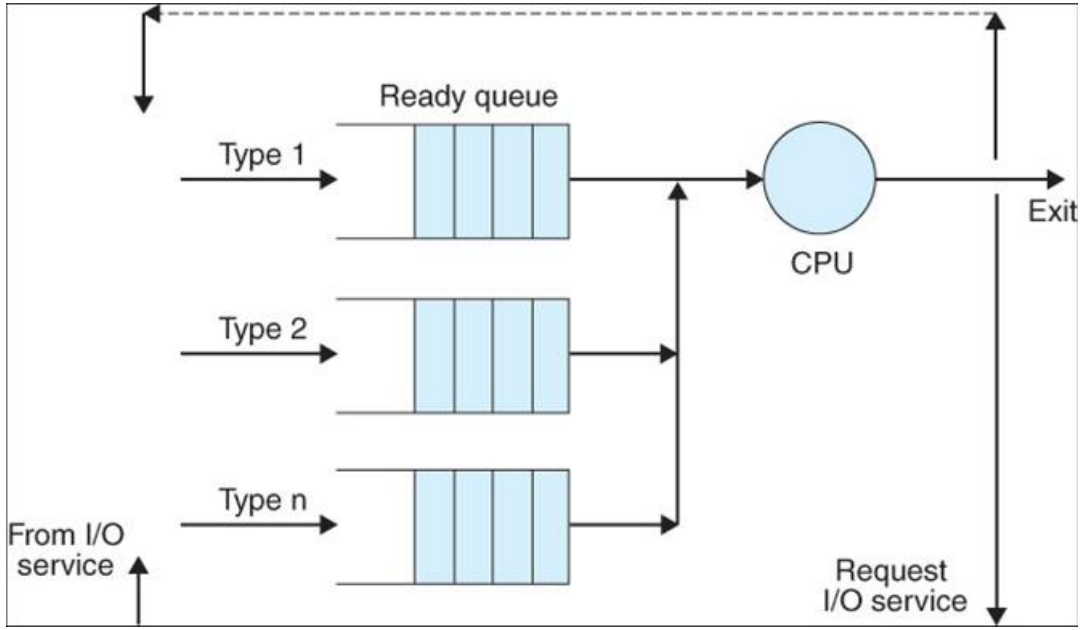
- Bu process'ler farklı response-time gerekliliklerine sahiptir.

- Process zamanlamada öncelikleri birbirinden farklı process'ler için tek düzeyli hazır kuyruğu (ready queue) yetersiz kalmaktadır.

- Bu probleme çözüm olarak çok düzeyli hazır kuyruğu (ready queue) kullanılmaktadır.

# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)




Tek ve Çok Düzeyli Hazır Kuyruk (Ready Queue) Yapısı

# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

- Çok düzeyli kuyruk zamanlama algoritması hazır kuyruğunu (ready queue) birkaç ayrı kuyruğa böler.

- Her process bu kuyruklardan birine sürekli olarak dahil olmaktadır.

- Processler, genellikle bellek boyutuna, process önceliğine veya process türü gibi process'in bazı özelliklerine dayalı olarak bir kuyruğa atanır.

# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

- Process kuyruklarının da kendi aralarında öncelik durumları vardır.

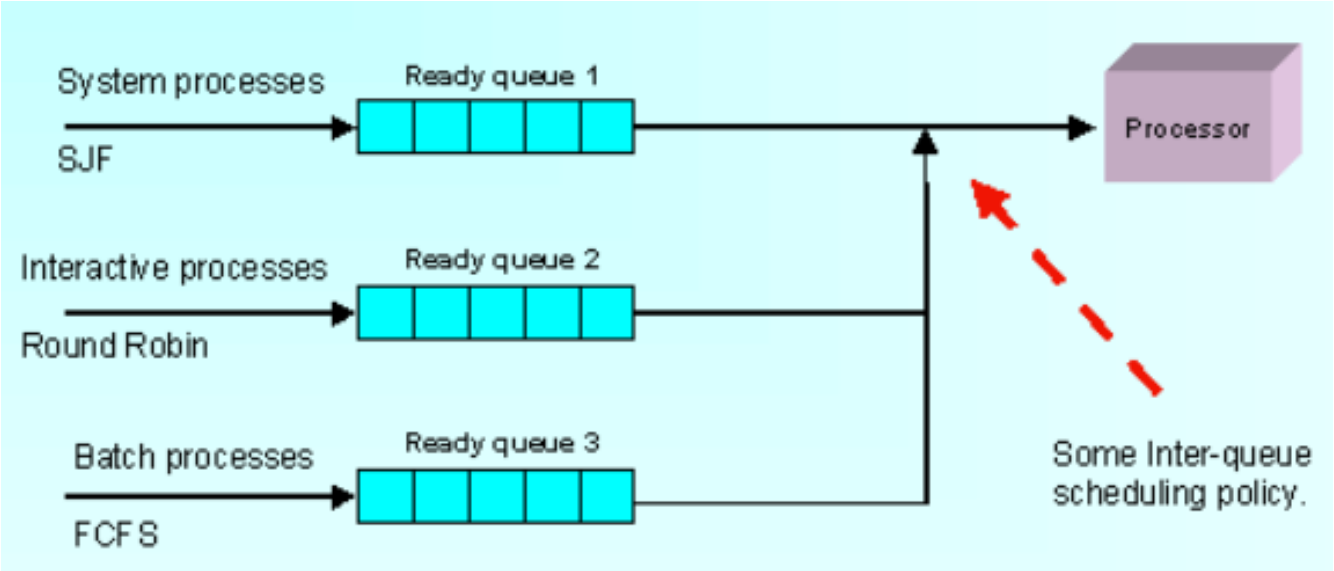
- Her kuyruk düşük öncelikli kuyruğa göre mutlak bir önceliğe sahiptir.
- Çok Düzeyli Kuyruk Zamanlamada;
- Process, belirli bir kuyruğa kalıcı olarak atanır ve kuyruklar arasında yer değiştirme olayı olmaz.

# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

Farklı process türleri farklı zamanlama gereksinimlerine sahip olabilir.
Bu yüzden her kuyruk kendi zamanlama algoritmasına sahip olabilir.

Örneğin ;
Sistem process'leri (System processes) Shortest Job First (SJF) ve Etkileşimli process'ler (Interactive processes) , Round Robin (RR) algoritması kullanılarak zamanlanırken,

Arkaplan process'leri (Background processes), First-Come First-Served (FCFS) algoritması kullanılarak zamanlanabilir.




Çok Düzeyli Kuyruk (Multilevel Queue )

# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

- Kuyruklar arasında da zamanlama yapılmalıdır.
- Bu iki yöntemle yapılır:

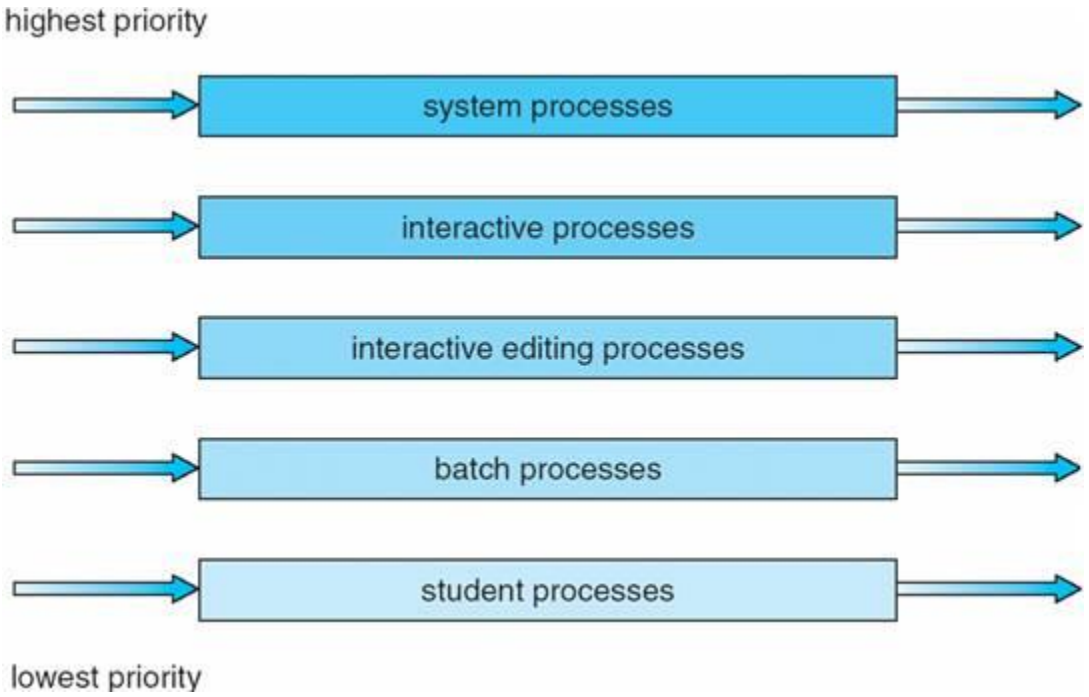
1. **Sabit öncelikli kesintili zamanlama**
   **(Fixed priority preemptive scheduling)**

2. **Zaman paylaşımı**
   **(Time slicing)**

# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

**1. Sabit öncelikli kesintili zamanlama (Fixed priority preemptive scheduling)**

- Her kuyruğun daha alt düzey kuyruğa göre önceliği bulunmaktadır.
- (Örneğin, öncelikle system processes kuyruğundaki process'leri yürüt daha sonra interactive processes kuyruğundaki process'leri yürüt, .....)



# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

**1. Sabit öncelikli kesintili zamanlama (Fixed priority preemptive scheduling)**

Her kuyruk, düşük öncelikli kuyruk üzerinde mutlak önceliğe sahiptir.

Öncelikli kuyruk sıralamasının **kuyruk1 > kuyruk2 > kuyruk3** şeklinde olduğunu varsayalım.

Bu algoritmaya göre, kuyruk3 deki hiçbir process, kuyruk1 ve 2 boş olmadıkça işlemciye gönderilemez.

Eğer kuyruk3 den gelen herhangi bir process CPU'da yürütülüyorsa, bu anda herhangi bir process kuyruk1 veya kuyruk2'de sıraya girerse, kuyruk3'ten gönderilen process kesintiye uğratılır (preempted).

# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

## 1. Sabit öncelikli kesintili zamanlama (Fixed priority preemptive scheduling)

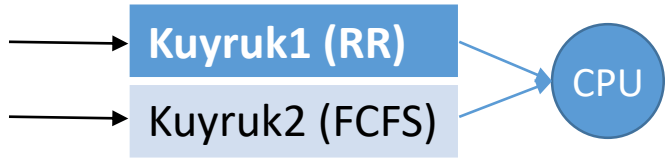
- Bir process işlemciyi kullanırken, daha yüksek öncelikli kuyruğa bir process geldiğinde, mevcutta işlemcide yürütülen düşük öncelikli kuyruktan gelen process kesilir ve yeni yüksek öncelikli process işlemcide yürütülür.

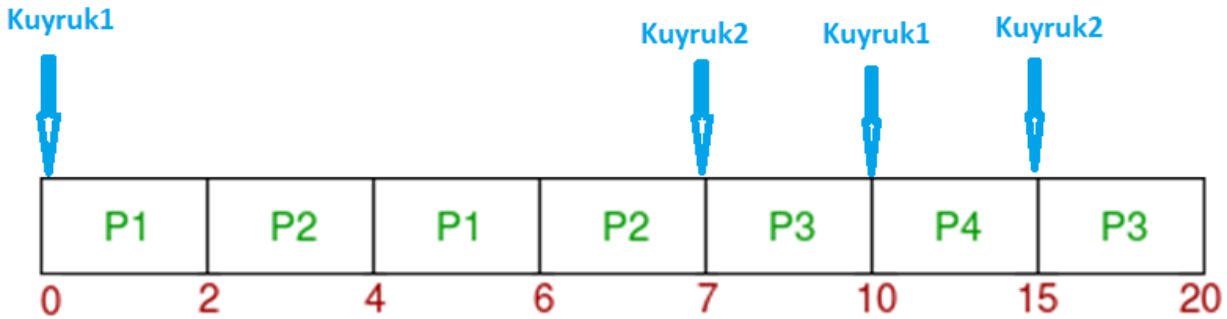
- Yani alt seviyedeki bir kuyruktan gelen process işlemcide yürütülürken, üst seviyedeki kuyruğa bir process geldiğinde mevcut process'in işletimi yarıda kesilir ve yeni gelen process işletime alınır.

# Örnek : Sabit öncelikli kesintili zamanlama (Fixed priority preemptive scheduling)

- Çok düzeyli kuyruk zamanlama ile aşağıdaki tabloda verilen dört process'i göz önüne alalım. Tablodaki kuyruk numarası (queue number) hangi process'in hangi kuyrukta bulunduğunu ifade etmektedir. Kuyruk 1'in önceliği, kuyruk 2'den büyüktür. Kuyruk 1, Round Robin (RR) (Time Quantum = 2) kullanmakta ve kuyruk 2 ise First-Come First-Served (FCFS) algoritmasını kullanmaktadır. Buna göre process'lerin işletimi için Gantt diyagramı ne şekilde olur?

| Process | Arrival Time (Kuyruğa varış zamanı) | CPU Burst Time (İşletim süresi) | Queue Number (Kuyruk numarası) |
|---|---|---|---|
| P1 | 0 | 4 | 1 |
| P2 | 0 | 3 | 1 |
| P3 | 0 | 8 | 2 |
| P4 | 10 | 5 | 1 |



# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

- **1. Sabit öncelikli kesintili zamanlama (Fixed priority preemptive scheduling)**
- **Dezavantaj :** En düşük seviyedeki kuyrukta bulunan process'ler için **Açlık (starvation)** problemi oluşur.

- **Açlık (starvation) :** Çok fazla etkileşimli process varsa veya sık sık yeni process'ler oluşturulmuşsa, her zaman yüksek öncelikli kuyrukta bir process olacaktır. Yüksek öncelikli kuyruk boşalmadıkça, düşük öncelikli kuyruktaki pcocess'ler asla işlemcide yürütülemez.
- Yada yüksek öncelikli kuyruk process'lerinin çok fazla zaman alması nedeniyle düşük öncelikli kuyruktaki pcocess'ler uzun süre beklemede kalır.

# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

## 2. Zaman paylaşımı (Time slicing) :

- Kuyruklar arasında işlemci kullanımı zamanlaması için diğer bir yöntem ise kuyruklar arasındaki zaman paylaşımıdır.
- Düşük seviyedeki kuyrukta bulunan process'ler için **Açlık (starvation)** probleminin oluşmaması için zaman paylaşımı yöntemi tercih edilir.
- Bu yöntemde işlemcide process yürütülebilmesi için her kuyruğa belirli bir zaman dilimi tahsis edilir.
- Bu zaman dilimi içinde kuyruktaki process'ler arasında işlemci (CPU) paylaştırılır.

# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

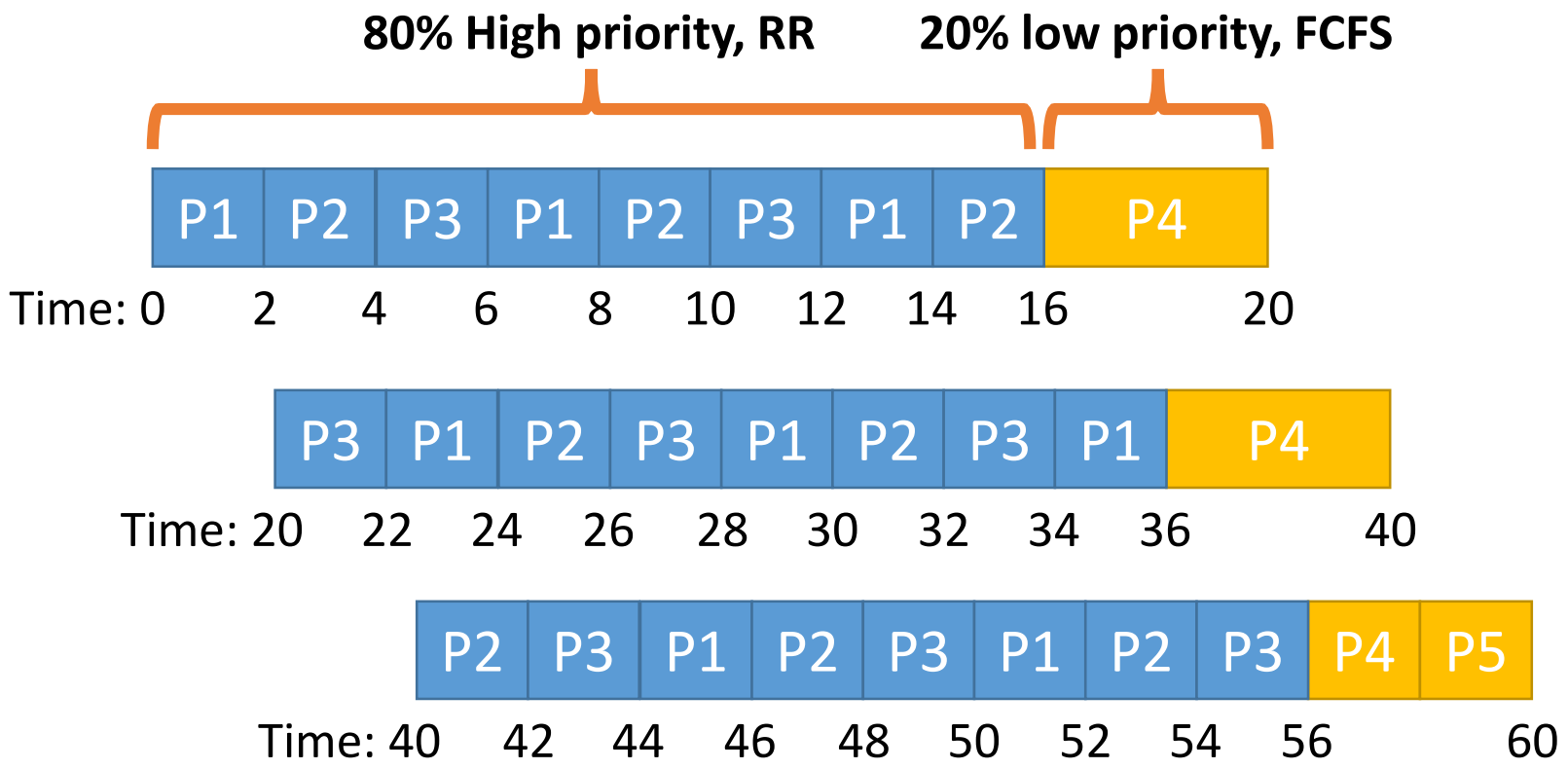
- **2. Zaman paylaşımı (Time slicing) :**
- Her kuyruğa ayrı ayrı sabit bir işlemci (CPU) süresi tahsis edilir.
- Burada, her kuyruk işlemci zamanının belli bir bölümünü kullanır;

- Örneğin, Yüksek öncelikli etkileşimli process'ler (Interactive processes) kuyruğuna, *Round Robin* (RR) zamanlaması için işlemci süresinin %80'i verilebilirken,

- Düşük öncelikli arkaplan process'leri (Background processes) kuyruğuna *First Come First Served* (FCFS) zamanlaması için işlemci süresinin %20'si kalır.

# Çok Düzeyli Kuyruk Zamanlama (Multilevel Queue Scheduling)

## 2. Zaman paylaşımı (Time slicing) :




| Process | Arrival Time | Priority |
|---|---|---|
| P1 | 0 | 1 |
| P2 | 0 | 1 |
| P3 | 0 | 1 |
| P4 | 0 | 2 |
| P5 | 1 | 2 |

# Çok Düzeyli Geri Beslemeli Kuyruk Zamanlama

# (Multilevel Feedback Queue Scheduling)

# Çok Düzeyli Geri Beslemeli Kuyruk Zamanlama (Multilevel Feedback Queue Scheduling)

- Geleneksel, çok düzeyli kuyruk zamanlama algoritması (Multilevel Queue Scheduling) kullanıldığında, process'ler, bir kuyruğa kalıcı olarak atanır.

- Örneğin, etkileşimli process'ler (interactive processes) ve arkaplan process'leri (background processes) için ayrı kuyruklar varsa, processler bir kuyruktan diğerine geçmez.

- Bu durum, düşük zamanlama yükü avantajına sahiptir, ancak esnek değildir.

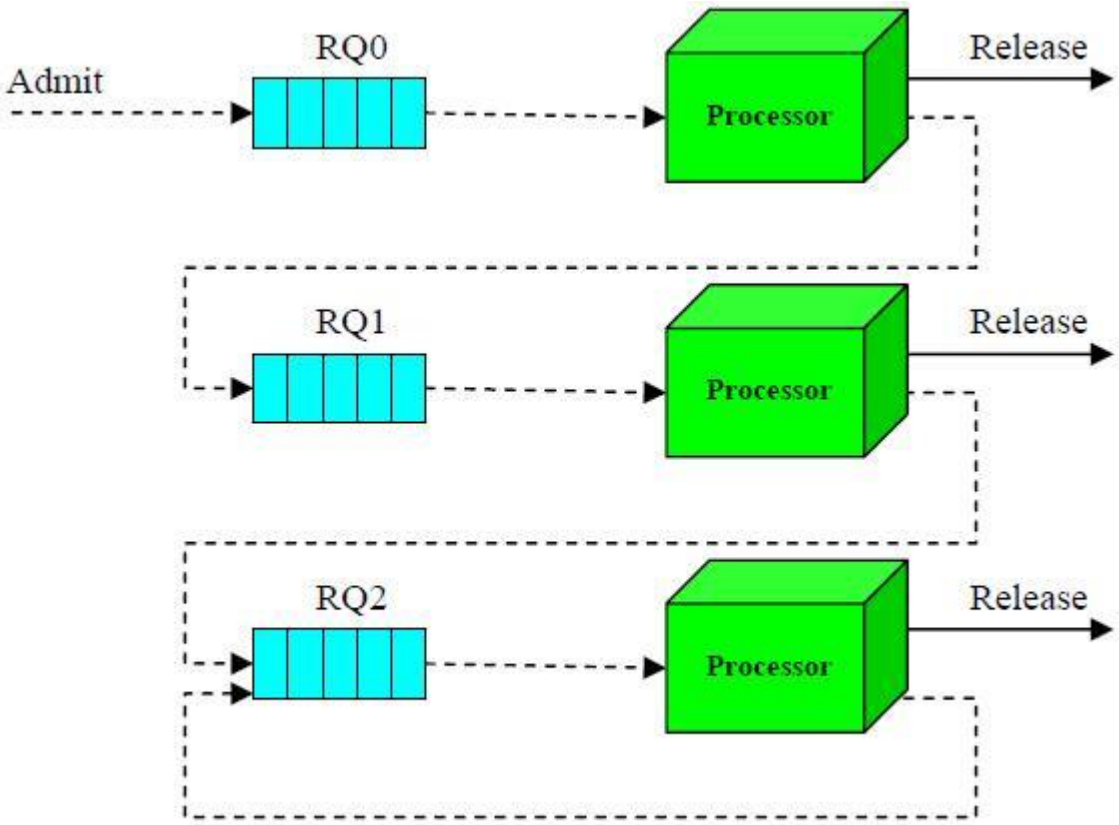
# Çok Düzeyli Geri Beslemeli Kuyruk Zamanlama (Multilevel Feedback Queue Scheduling)

- Buna karşılık, çok düzeyli **geri beslemeli** kuyruk zamanlama algoritması, bir process'in kuyruklar arasında hareket etmesine olanak tanır.

- Hazır kuyruğuna gelen process, öncelikle en yüksek öncelikli kuyruğa (en üst kuyruk) alınır.

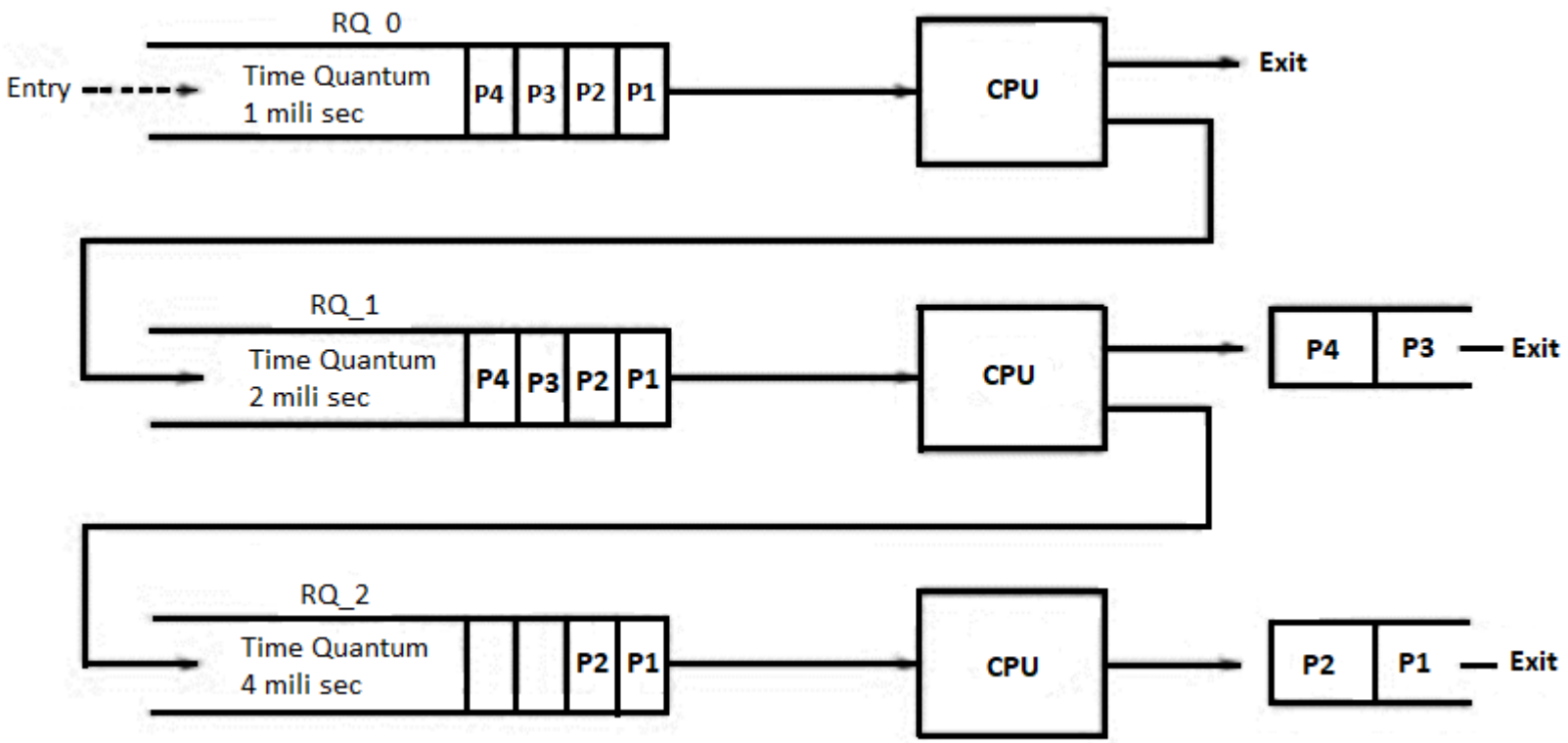
- Bir kuyruktan gelen process, işlemcide yürütülürken quantum zamanının tamamını kullanmışsa, önceliği azaltılır (diğer bir deyişle bir alt kuyruğa iner).

# Çok Düzeyli Geri Beslemeli Kuyruk Zamanlama (Multilevel Feedback Queue Scheduling)




Çok Düzeyli Geri Beslemeli Kuyruk

# Çok Düzeyli Geri Beslemeli Kuyruk Zamanlama (Multilevel Feedback Queue Scheduling)

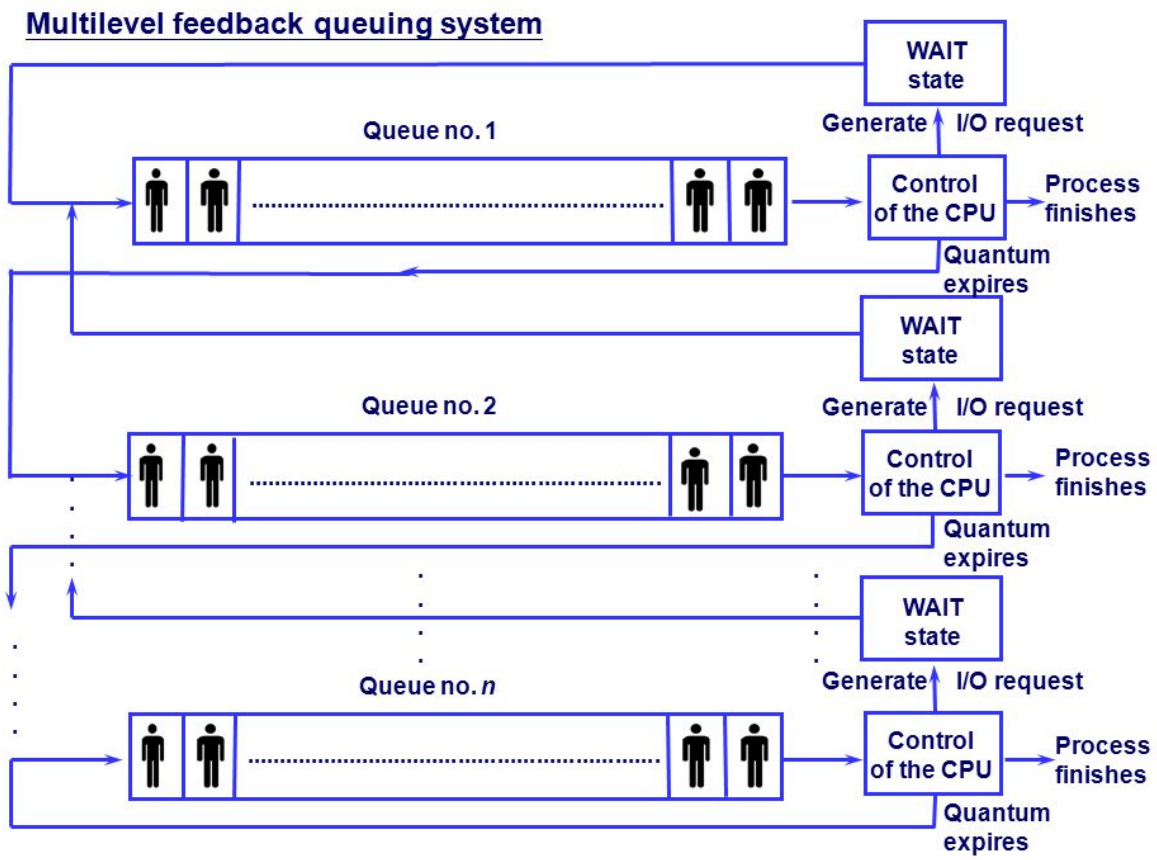



- Çok Düzeyli Geri Beslemeli Kuyruk yapısında process'ler ilk olarak zaman dilimi (quantum time) az olan en yüksek öncelikli kuyruğa konur.
- Bu zaman diliminde process tamamlanmazsa zaman dilimi biraz daha fazla olan bir alt kuyruğa aktarılır.

# Çok Düzeyli Geri Beslemeli Kuyruk Zamanlama (Multilevel Feedback Queue Scheduling)

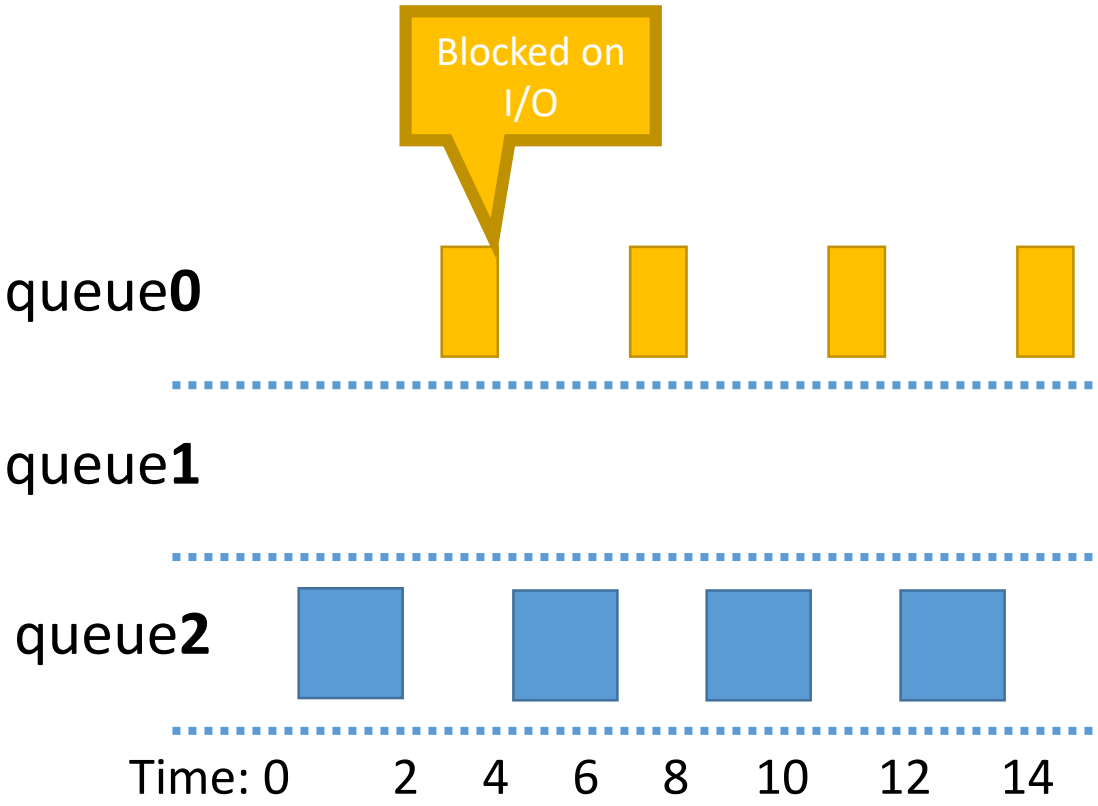
**Bir process quantum zamanı dolmadan bir giriş/çıkış isteği için (I/O request) işlemcide yürütülmesi kesilip işlemciyi bırakırsa, bir üst seviye öncelik düzeyindeki kuyruğa dahil olur.**

Bu düzen, Giriş/Çıkış bağımlı (I/O bound) process'leri yüksek öncelikli kuyruklar içinde bırakır.

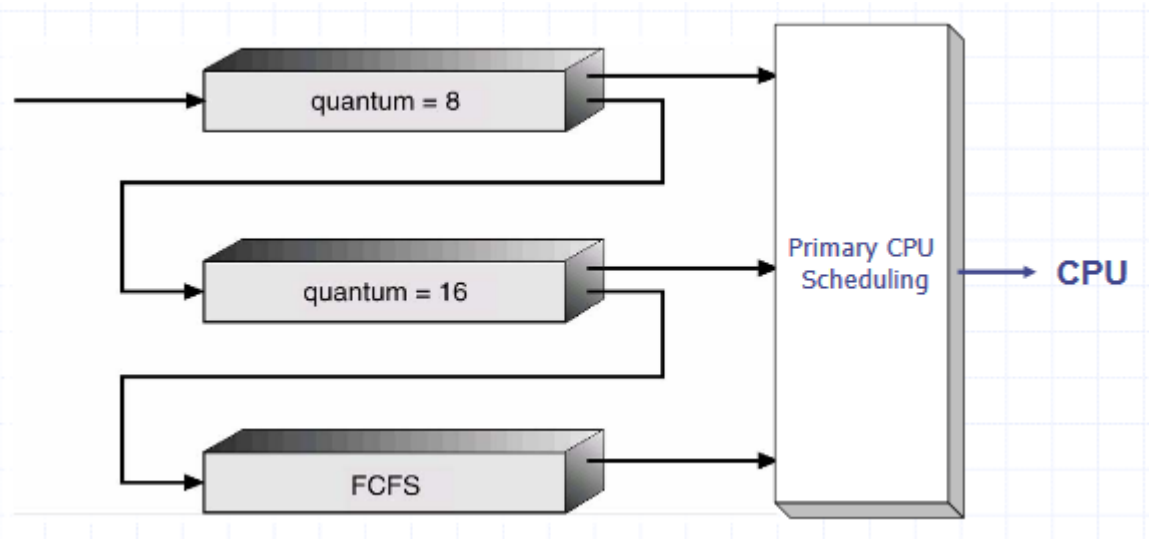


# Çok Düzeyli Geri Beslemeli Kuyruk Zamanlama (Multilevel Feedback Queue Scheduling)

**I/O Bound and CPU Bound Processes**



# Çok Düzeyli Geri Beslemeli Kuyruklar (Multilevel Feedback Queue)




Üç kuyruk:

Q0 – quantum time = 8 milisaniye

Q1 – quantum time = 16 milisaniye

Q2 – FCFS

Bir process Q0 kuyruğuna girer. Process CPU'yu ele geçirdiğinde 8 ms süre verilir, bu process 8 ms'de bitmezse process Q1 kuyruğuna gider.

Q1 kuyruğunda 16 ms daha hizmet görür.

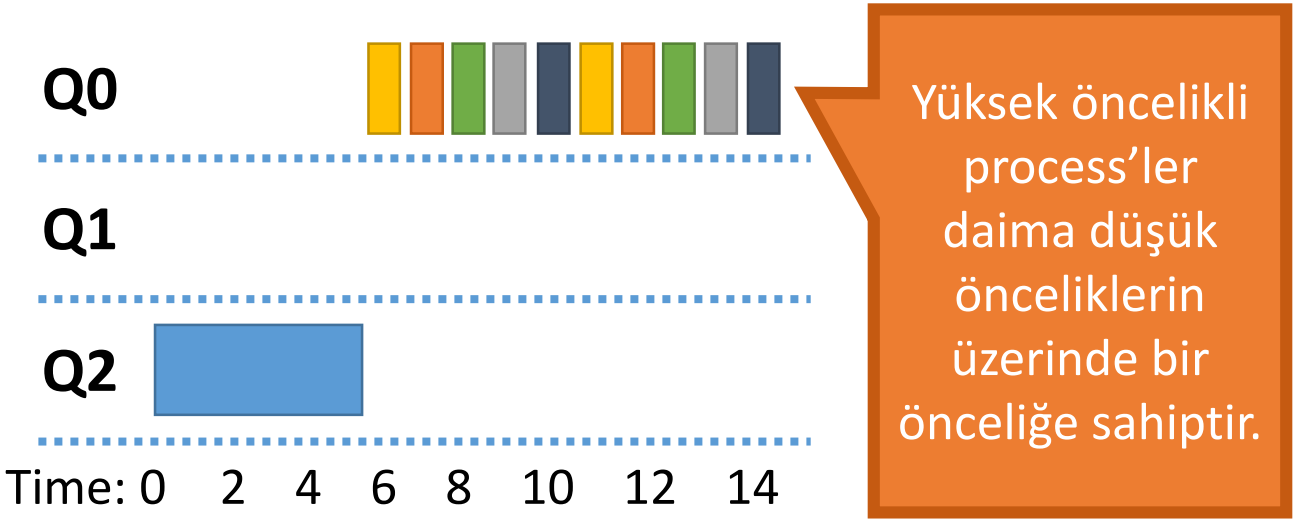
Eğer process bu süre ile hala tamamlanamazsa daha sonra Q2 kuyruğuna gönderilir ve burada FCFS yöntemiyle zamanlanır.

(Not: Üst iki seviyede, process'ler kuyruğa geldikleri sırada işletime alınmaktadır.)

# Çok Düzeyli Geri Beslemeli Kuyruklar (Multilevel Feedback Queue)

- Uygulamadaki problem :
- Açlık (Starvation)

- Düşük öncelik kuyruğundaki bir process, tüm işlemci zamanlarını alan sık sık yeni oluşturulan kısa process'lerin yürütülmesinden dolayı açlık (starvation) problemi ile karşılaşabilir.




Her zaman yüksek öncelikli bir kuyrukta bir process var olduğu sürece, düşük öncelikli bir kuyruktaki process'ler zamanlanmayabilir. Process, düşük öncelikli kuyrukta uzun süre kalmaya mahkum olabilir.

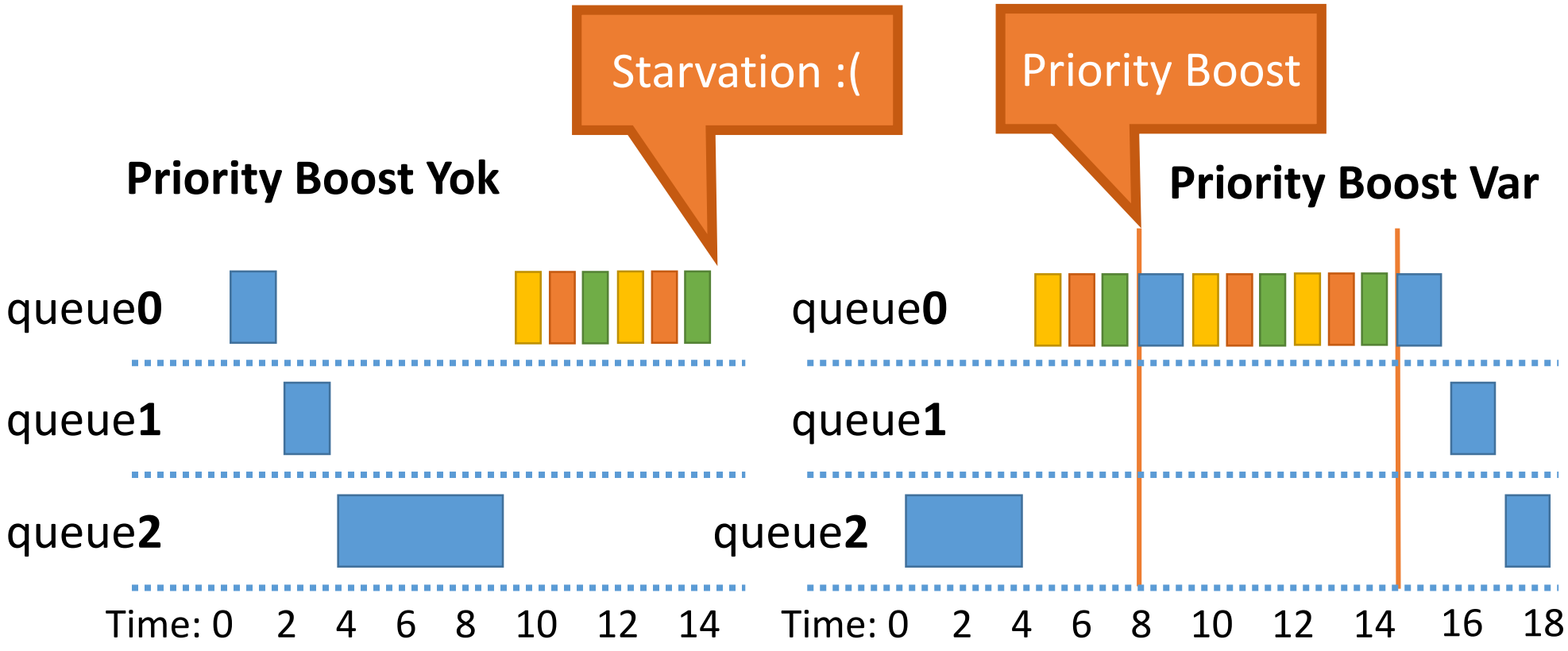
# Öncelik Artışı (Priority Boost)

- Açlık (Starvation) çözümü :
- Daha düşük öncelikli bir kuyrukta çok uzun süre bekleyen process'ler düzenli aralıklarla daha yüksek öncelikli kuyruğa taşınabilir.
- Bir zaman dilimi S'den sonra, tüm process'ler en yüksek öncelikli kuyruğa taşınır (Öncelik Artışı - **Priority Boost**).
- Bu şekilde Açlık (Starvation) problemine çözüm getirilmeye çalışılır.
- Düşük öncelikli kuyruklardaki process'ler sonunda öncelikli kuyruğa gelecek ve işlemci zamanını kazanacaktır.

# Öncelik Artışı (Priority Boost)

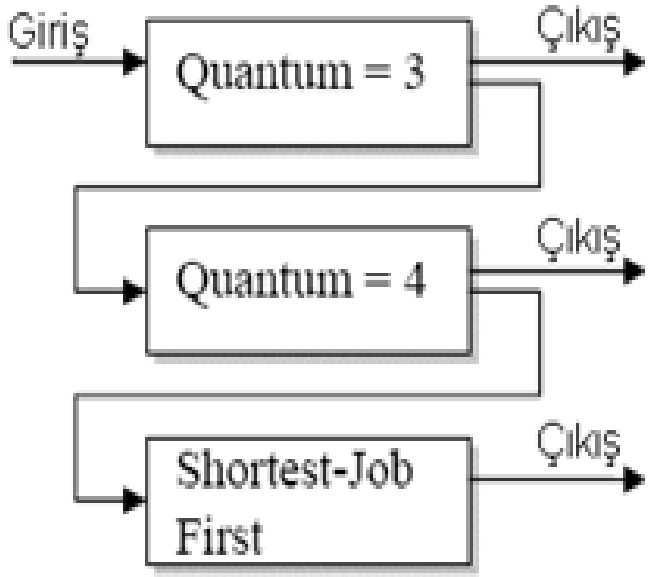
- Gerçek hayatta bir bankada sıra bekleyen kişiler düşünüldüğünde;
- Örneğin banka kendi müşterilerine 1. derece önceliği verirken, kredi işlemi yapan müşterilere ikinci ve diğer müşterilere 3. derece önceliği verebilir.
- Fakat bankaya sürekli olarak kendi müşterisi gelmesi durumunda, diğer müşterilere sıra asla gelmeyebilir.
- Bunun çözümü olarak, düşük dereceli seviyelerden, daha yüksek dereceli seviyelere belirli aralıklarla bazı kişilerin geçirilmesi gerekir.

# Öncelik Artışı (Priority Boost) Örneği



# Çok Düzeyli Geri Beslemeli Kuyruklar (Multilevel Feedback Queue)

- **Örnek:**
- Çok kuyruklu geri beslemeli zamanlama algoritmasının kullanıldığı bir sistemde, process'lerin kuyruğa geliş zamanları ve işletim süreleri aşağıdaki tabloda gösterildiği gibidir. Process'lerin ne şekilde işletileceğini Gantt diyagramı ile gösteriniz.
- (Not: Üst iki sevide, process'ler kuyruğa geldikleri sırada işletime alınmaktadır.)

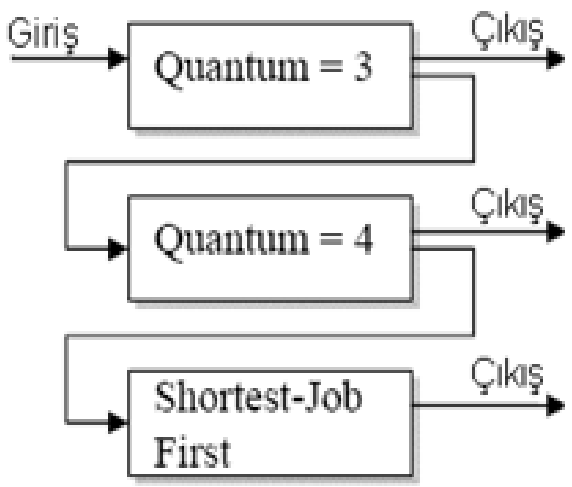


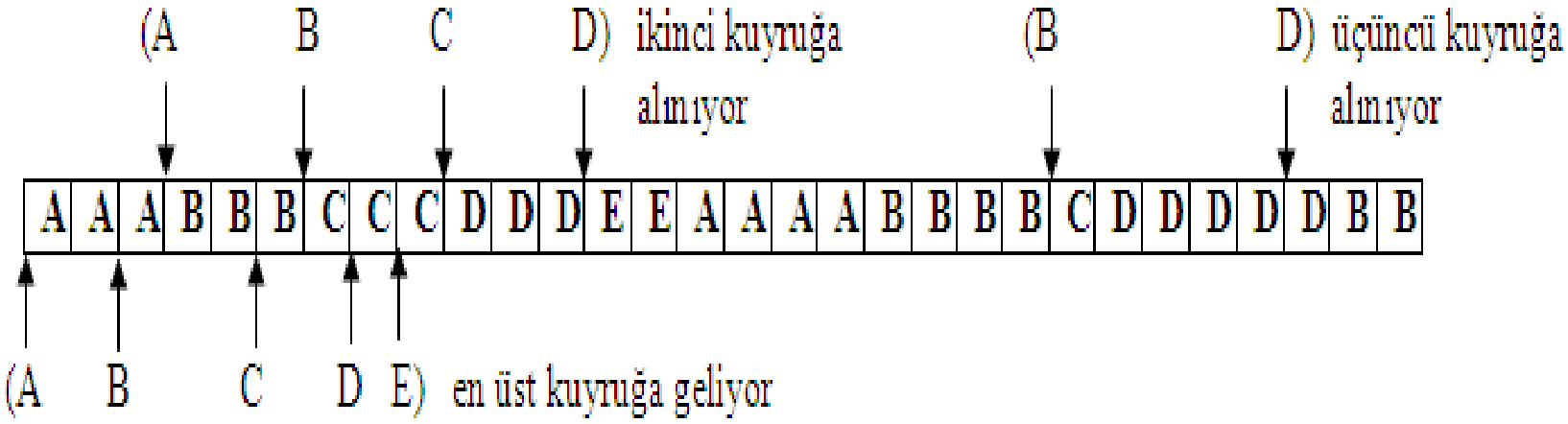

| Process ID | Varış zamanı (arrival time) | İşletim Süresi (Burst time) |
|---|---|---|
| A | 0 | 7 |
| B | 2 | 9 |
| C | 5 | 4 |
| D | 7 | 8 |
| E | 8 | 2 |

# Çok Düzeyli Geri Beslemeli Kuyruklar (Multilevel Feedback Queue)

- ÇÖZÜM

| Process ID | Varış zamanı (Arrival time) | İşletim Süresi (Burst time) |
|---|---|---|
| A | 0 | 7 |
| B | 2 | 9 |
| C | 5 | 4 |
| D | 7 | 8 |
| E | 8 | 2 |



# Çok İşlemcili Zamanlama (Multiple-Processor Scheduling)

- Çok işlemcili sistemler, birden fazla işlemciye (veya işlemci çekirdeğine) sahiptir ve aynı anda birden fazla process'i gerçek paralel çalıştırabilirler.
- Birden fazla işlemci mevcut olduğunda işlemci zamanlaması daha karmaşıktır.
- Çok işlemcili zamanlama (Scheduling) için tek bir 'en iyi' çözüm mevcut değildir.

- İki tür çok işlemcili zamanlama yaklaşımı mevcuttur:

- Asimetrik Çoklu İşlem (Asymmetric multiprocessing -AMP)
- Simetrik Çoklu İşlem (Symmetric multiprocessing -SMP)

# Çok İşlemcili Zamanlama (Multiple-Processor Scheduling)

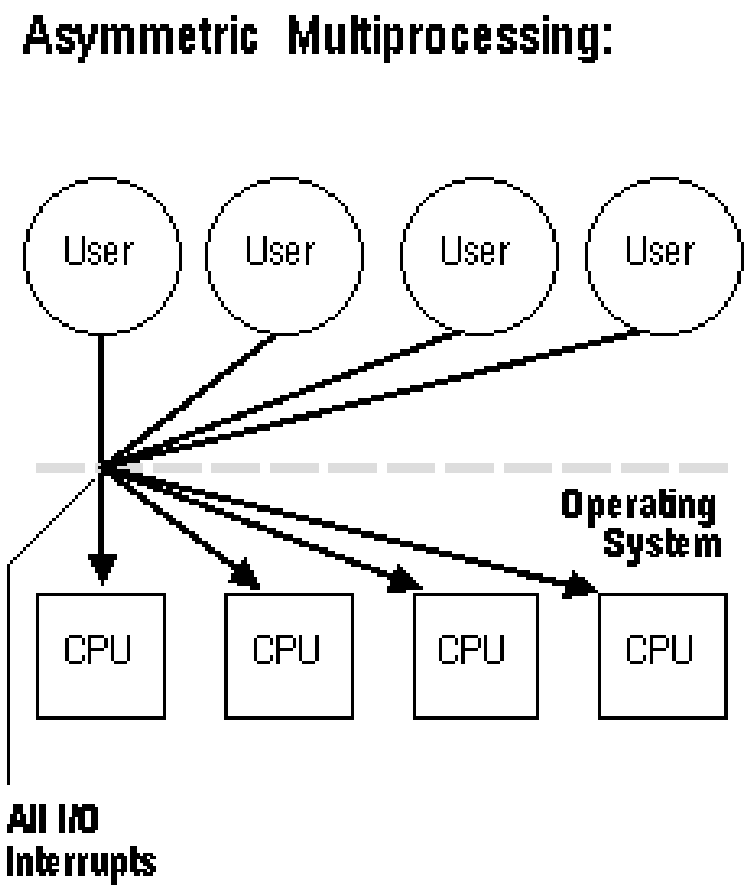
- Simetrik ve asimetrik çoklu işlem arasındaki en ayırt edilebilir noktalar;

- Asimetrik Çoklu İşlem(Asymmetric multiprocessing -AMP):
- Bu yaklaşımda, tüm zamanlama kararları ve G/Ç işlemleri, Master işlemci tarafından gerçekleştirilir ve diğer Slave işlemciler yalnızca kullanıcı kodunu yürütür.

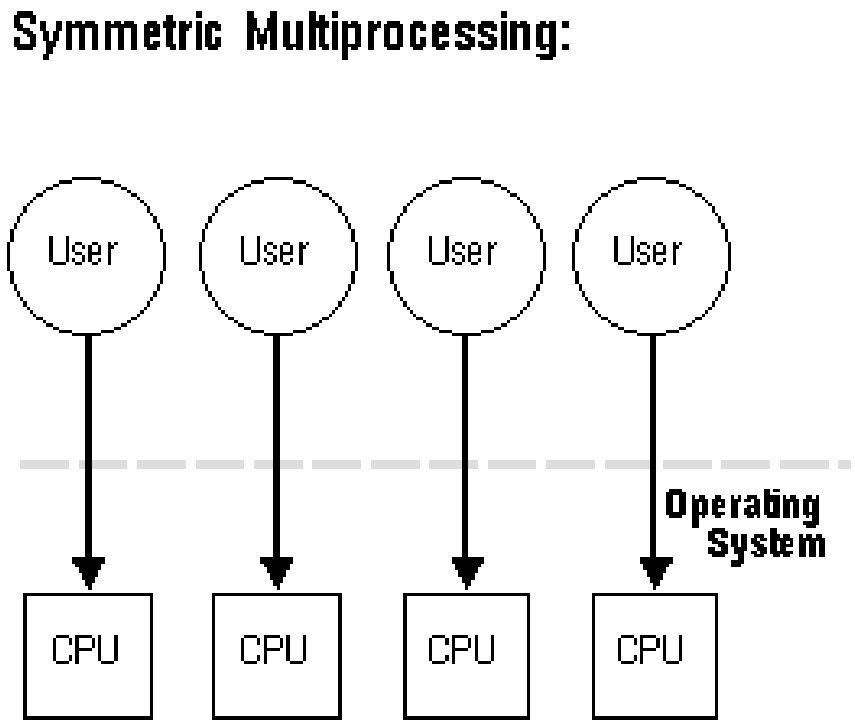
- Simetrik Çoklu İşlem (Symmetric multiprocessing -SMP):
- Bu yaklaşımda, her işlemci kendi process zamanlamasını ve G/Ç işlemlerini yapar. Tüm process'ler ortak bir hazır kuyruğuna sahip olabilir veya her işlemcinin hazır process'ler için kendi özel kuyruğu olabilir.

- Simetrik çoklu işlem şu anda, çok işlemcili zamanlama için en yaygın kullanılan yaklaşımdır.

# Çok İşlemcili Zamanlama
# (Multiple-Processor Scheduling)

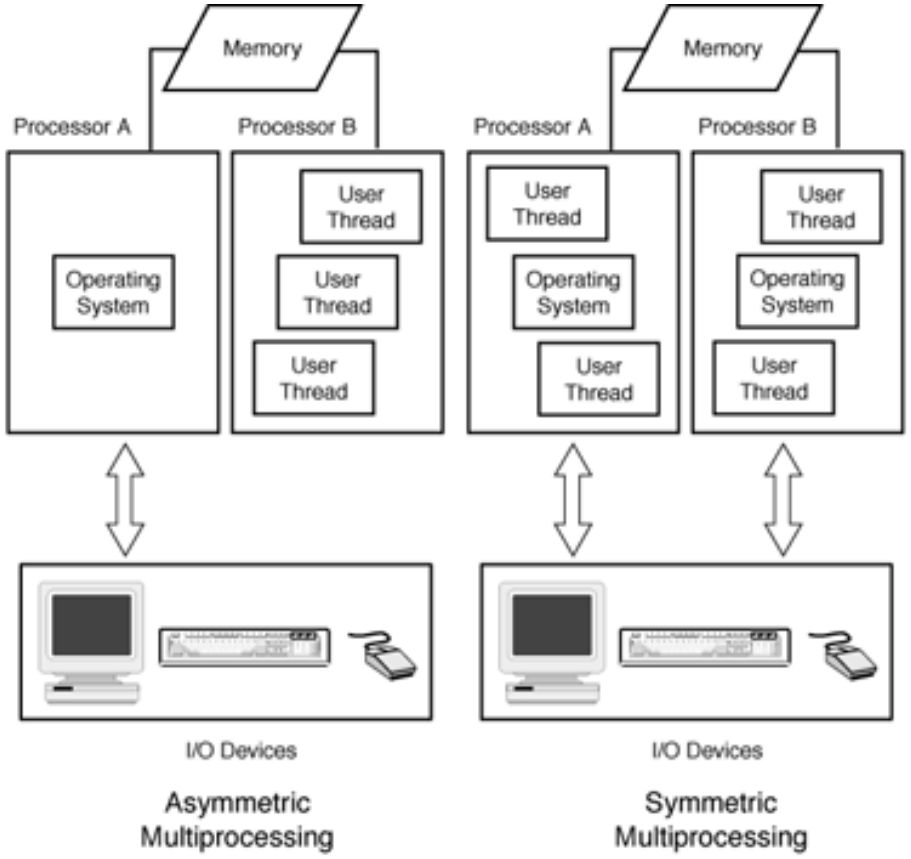



Asimetrik Çoklu İşlem
(Asymmetric multiprocessing -AMP)




Simetrik Çoklu İşlem
(Symmetric multiprocessing -SMP)

# Çok İşlemcili Zamanlama (Multiple-Processor Scheduling)



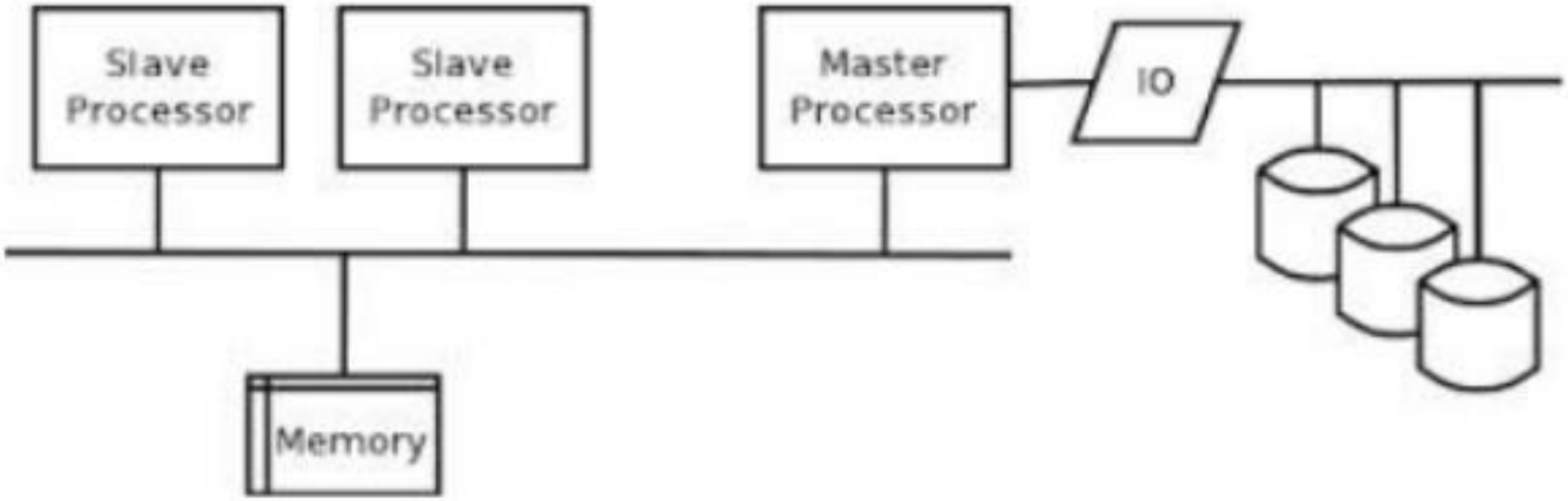

Asimetrik Çoklu İşlem
(Asymmetric multiprocessing - AMP)

Simetrik Çoklu İşlem
(Symmetric multiprocessing -SMP)

# Asimetrik Çoklu İşlem (Asymmetric multiprocessing)

- Asimetrik Çoklu İşlemde, işlemciler kendi arasında master-slave ilişkiye sahiptir.
- Slave işlemciyi denetleyen bir master işlemci vardır. Master işlemci, process'leri slave işlemcilere dağıtır.
- Asimetrik Çoklu İşlemdeki tüm işlemcilerin mimarileri aynı veya farklı olabilir.
- Bir master işlemci başarısız olursa, çalışmaya devam etmek için bir slave işlemci master işlemci konumuna getirilir.
- Bir slave işlemci başarısız olursa, görev diğer slave işlemcilere geçirilir.

# Asimetrik Çoklu İşlem (Asymmetric multiprocessing)




- Process'lerin zamanlaması, G/Ç işlemi ve diğer sistem faaliyetleri master işlemci tarafından kontrol edilir.

# Asimetrik Çoklu İşlem (Asymmetric multiprocessing)

- G/Ç işlemlerinin ve diğer sistem tabanlı process'lerin tek bir işlemcide, kullanıcı process'lerin ise diğer işlemciler üzerinde çalıştırıldığı yöntemdir.

- Bu yöntemde, sadece bir işlemci sistem verilerine erişmekte ve verilerin paylaşımını en aza indirmektedir.

- Asimetrik Çoklu İşlem basittir, çünkü veri yapısını ve sistemdeki tüm faaliyetleri kontrol eden yalnızca bir işlemci bulunur.

# Simetrik Çoklu İşlem
# (Symmetric multiprocessing -SMP)

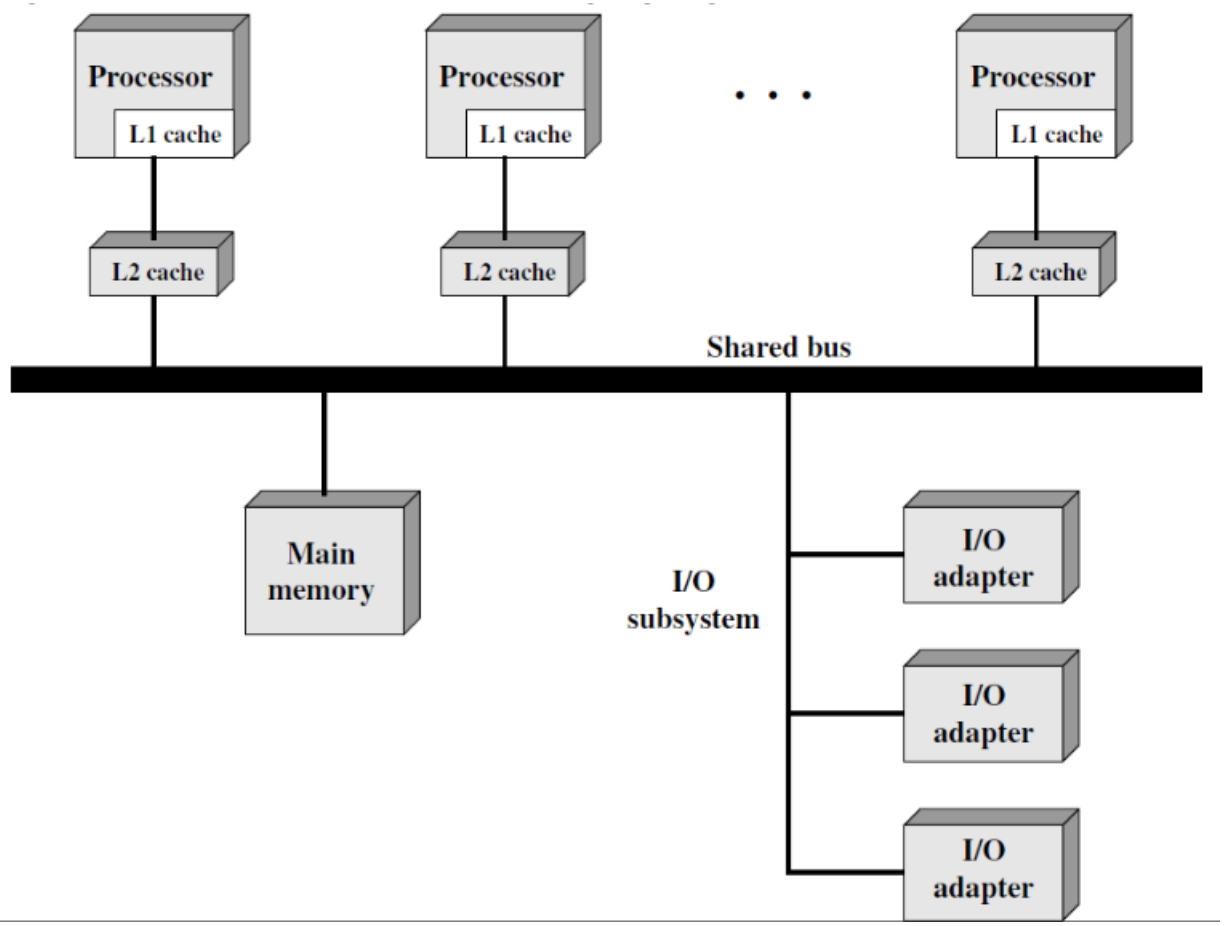
- Simetrik Çoklu İşlem, uygun yük dengeleme özelliğine ve daha iyi hata toleransına sahiptir.
- Bellek, tüm işlemciler arasında paylaşıldığından karmaşıktır.
- Simetrik Çoklu İşlemde bir işlemci arızası, hesaplama kapasitesinin azalmasına neden olur.

# Simetrik Çoklu İşlem
# (Symmetric multiprocessing -SMP)

- Simetrik Çoklu İşlemdeki tüm işlemciler aynı mimariye sahiptir.
- Bütün process'ler aynı hazır kuyruğunda olabileceği gibi her bir işlemcinin kendine özel hazır kuyruğu da olabilir.
- Her işlemci kendi özel hazır kuyruğunu kontrol edebilir.
- Modern işletim sistemleri Simetrik Çoklu İşlemi destekler.

# Simetrik Çoklu İşlem (Symmetric multiprocessing -SMP)

- SMP çok işlemcili sistemler zaman paylaşımlı bus kullanır.



# Simetrik Çoklu İşlem
# (Symmetric multiprocessing -SMP)

- Simetrik çoklu işlemde, işlemciler paylaşımlı bellek kullandıklarından birbirleriyle iletişim içinde olmalıdırlar.

- Fakat process'lerin bir işlemciden diğerine alındığı durumlarda işlemci üzerindeki process'e özel bilgiler sorun oluşturmaktadır.

# Simetrik Çoklu İşlem (Symmetric multiprocessing -SMP)

- Bir process başka bir işlemciye aktarıldığında eski işlemcideki önbellek (cache) verileri aktarılmaz.
- Yeni aktarılan işlemcinin önbellek verileri oluşana kadar hit rate oranı düşük kalır.
- Bu nedenle birçok Simetrik Çoklu İşlem sisteminde process'lerin çalışmaya başladıkları işlemciler çoğu zaman değiştirilmez.

# İşlemci akrabalığı (Processor affinity)

- İşlemci akrabalığı (**processor affinity**), bir işletim sisteminin yürütülecek process'ler açısından bir bilgisayarın işlemci kaynaklarını kullanma şeklini kontrol etmesine izin veren bir kavramdır.

- Processler genelde yürütüldükleri işlemcilerle ilişkilendirilir ve bir process'in belirli bir işlemcide yürütülmesine dikkat edilir, buna işlemci akrabalığı denir.

# İşlemci akrabalığı (Processor affinity)

- **İşlemci akrabalığı neden yararlı?**

- Bir process sık sık çalışırsa, belirli bir işlemci önbelleğinde bu process'in önceki veri kalıntıları olabilir.

- Dolayısıyla, process her zaman aynı işlemci çekirdeğini kullanmak için yönlendirilirse, process'in önbelleği yeniden kullanımı nedeniyle daha verimli ve hızlı çalışabilmesi mümkündür.

# İşlemci akrabalığı (Processor affinity)

- **İşlemci akrabalığını kullanmanın başka bir yararı;**

- Yalnızca bir veya iki performans kritik process'i bir işlemciye yönlendirirken, diğer tüm process'lerin diğer işlemcilere yönlendirilmesidir.

- Bu durum, performans kritik process'lerin bir işlemci gerektirdiğinde mevcutta olmasını sağlayacaktır.

# İşlemci akrabalığı (Processor affinity)

- Bununla birlikte, yanlış yapılandırılmış bir işlemci akrabalığı, sistem performansını olumsuz etkileyebilir.
- Örneğin, işlemci akrabalığını aşırı derecede sınırlandırmak, iş yükünün yavaş çalışmasına ve sistem performansının düşmesine neden olabilir.
- Bu nedenle, işlemci akrabalığının doğru şekilde ayarlanması önemlidir.

# İşlemci akrabalığı (Processor affinity)

- İşlemci akrabalığı iki türlüdür:

- **Zayıf işlemci akrabalığı (Soft affinity)**

- **Güçlü işlemci akrabalığı (Hard affinity)**

# Zayıf işlemci akrabalığı (Soft affinity):

- Soft affinity, iş yükünün ne zaman hangi işlemci veya işlemci çekirdeğine yönlendirileceğine dair esnekliği sağlar.
- Process'lerin başka işlemciye aktarılmayacağının temel olduğu ancak aktarımın olabileceği durumdur.
- Process'ler mümkün olduğu kadar aynı işlemci üzerinde tutulmaya çalışılır. Eğer bu mümkün değilse, process'ler başka bir işlemciye geçecektir.
- İşletim sistemi bir process'i aynı işlemcide çalışmaya devam etmeye çalışacaktır, ancak garanti yoktur.

# Güçlü işlemci akrabalığı (Hard affinity)

- İşlemciden işlemciye process aktarımının kesinlikle olmayacağı durumdur.

- İşletim sistemi, bir process'i bir işlemciye atar ve her zaman aynı işlemci üzerinde çalışmayı garanti eder.

- (Linux işletim sistemi, sistem çağrıları ile güçlü işlemci akrabalığı sağlar.)

# Yük Dengeleme (Load Balancing)

Yük Dengeleme, iş yükünün SMP (Symmetric multiprocessing) sisteminde tüm işlemciler arasında eşit olarak dağıtılması olayıdır.

Genellikle her işlemcinin kendine özel process kuyruğunun bulunduğu sistemlerde yük dengeleme gereklidir.

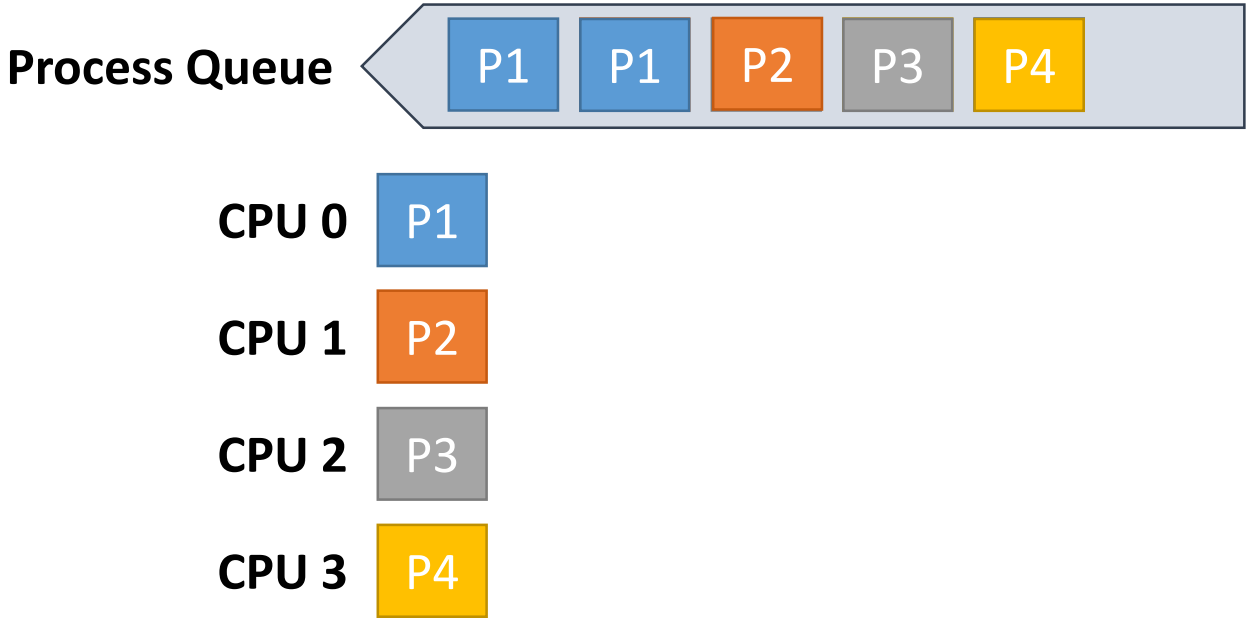
Bu sistemlerde yük dengeleme iyi yapılmazsa bazı işlemciler boşta beklerken diğer işlemciler yoğun çalışabilir.

<u>Ortak kuyruk kullanan sistemlerde yük dengelemeye ihtiyaç olmaz.</u>

# Yük Dengeleme (Load Balancing)

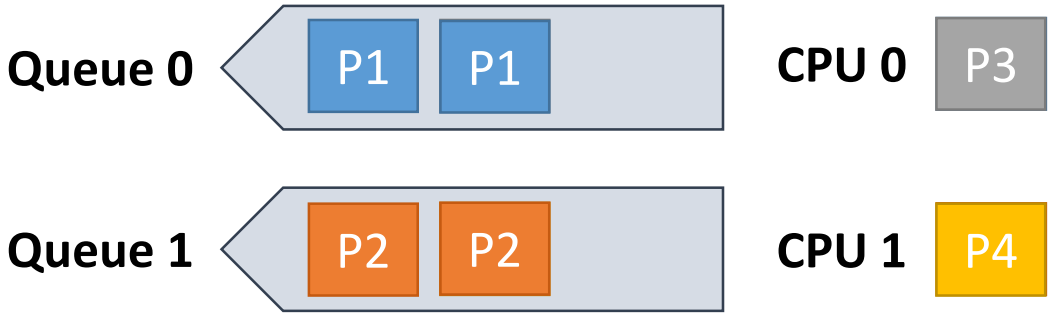
Tek kuyruklu sistemlerde, herhangi bir işlemci, process'leri ortak bir hazır kuyruğundan çekebilir.

Bu durumda işlemcinin boşta kalma durumu pek oluşmamaktadır.



# Yük Dengeleme (Load Balancing)

- Çoğu modern işletim sistemi, her işlemciye özel bir kuyruk atamaktadır.



# Yük Dengeleme (Load Balancing)

Yükün dengelenmesi iki yöntemle gerçekleştirilir;

1) **Push migration**
2) **Pull migration**

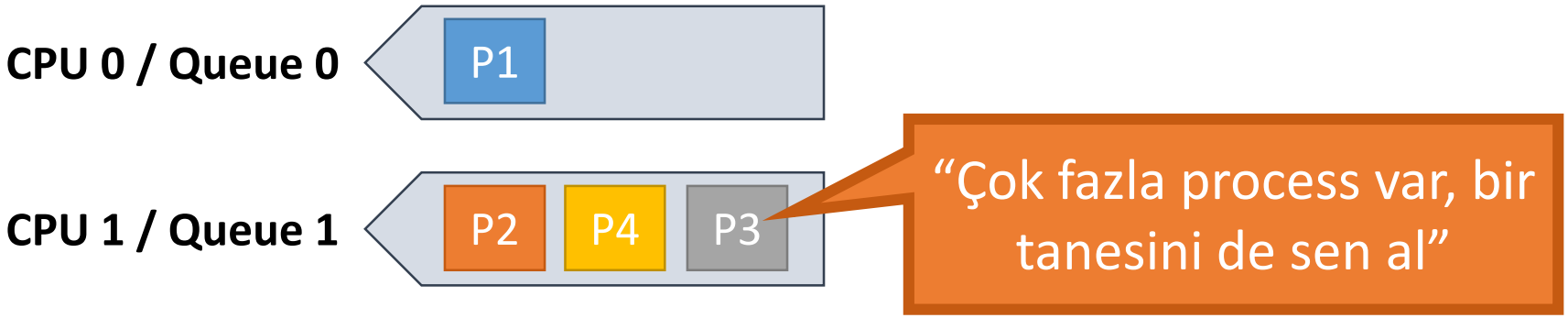
Yük dengelemesi, işlemci akrabalığının (processor affinity) avantajlarını olumsuz etkileyebilir.

# Yük Dengeleme (Load Balancing)

## 1) Push migration :

Özel bir process periyodik olarak her işlemci üzerindeki yükü (çalışma sırasındaki process sayısı) kontrol eder.

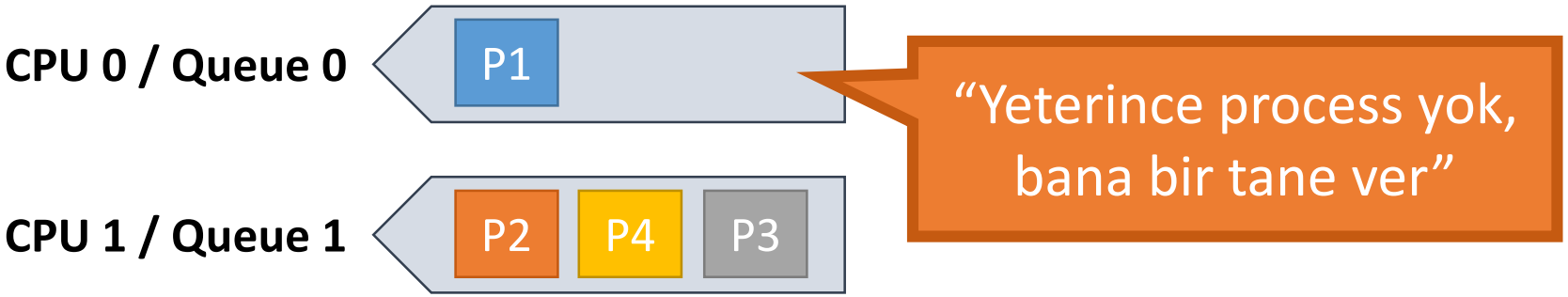
Dengesiz bir dağılım oluştuğunda hazır kuyruğunda process yükü fazla olan işlemcideki bazı process'ler boşta veya daha az yükü olan işlemcilere doğru itilir.



# Yük Dengeleme (Load Balancing)

## 1) Pull migration :

Zamanlayıcı bir işlemcinin hazır kuyruğunda az sayıda process olduğunu tespit ettiği durumda, yükü fazla olan başka bir işlemcinin hazır kuyruğundaki process'lerden bazılarını kendi kuyruğuna çeker.



# Çok çekirdekli İşlemciler (Multicore Processors)

- Son zamanlarda mimari eğilim, aynı fiziksel yonga üzerine birden fazla işlemci çekirdeği yerleştirmektir.

- Her bir çekirdek, mimari durumunu korumak için bir register setine sahiptir ve bu nedenle işletim sistemine ayrı bir fiziksel işlemci olarak görünür.

- Çok çekirdekli işlemcileri kullanan SMP sistemleri, daha hızlıdır ve her işlemcinin kendi fiziksel yongasına sahip olduğu sistemlerden daha az güç tüketir.

# Çok çekirdekli İşlemciler (Multicore Processors)

- Ancak çok çekirdekli işlemciler, programlama sorunlarını zorlaştırabilir.

- İşlemci belleğe eriştiğinde, verilerin kullanılabilir hale gelmesini beklemek için önemli miktarda zaman harcanır. Bu duruma MEMORY STALL denir.

- Bu sorunu çözmek için, multithreaded işlemci çekirdekleri geliştirilmiştir. Bu nedenle, bir thread bellek için beklerken, çekirdek başka bir thread için çalışabilir.

# Çok çekirdekli İşlemciler (Multicore Processors)

- SMP sistemleri, çoklu thread yapıların aynı anda birden fazla işlemci çekirdeği üzerinde koşturulmalarını sağlar.

- Intel işlemcilerinde hyper thread teknolojisi olarak bilinen simetrik çoklu thread çalışma ile thread'ler fiziksel işlemcilerin yanı sıra mantıksal işlemcilerde de yürütülebilir.

- Böyle yapılarda zamanlayıcılar process'leri mantıksal işlemciler yerine ilk olarak fiziksel işlemciler üzerinde paylaştırmalıdır.